Self-describing GOAL traces

Contents

1 Introduction **2**

2 Background **2**

2.1 GOAL schedules . . . 2

2.2 Workflow: generation, conversion and simulation . . . 3

2.3 Simulation . . . 3

2.4 The legacy binary format . . . 4

2.5 Related work . . . 5

3 Design and implementation **5**

3.1 Standard and custom fields . . . 6

3.2 Custom fields . . . 6

3.3 Inferring the layout . . . 6

3.4 The file format . . . 9

3.5 Reading through simulator adapters . . . 10

3.6 Runtime specialization . . . 13

4 Case study: explicit ATLAHS metadata **15**

4.1 Original encoding in standard fields . . . 15

4.2 Explicit GPU, stream and message fields . . . 16

4.3 Integration . . . 17

4.4 Remaining conventions . . . 19

5 Evaluation **19**

5.1 Correctness . . . 19

5.2 File size . . . 20

5.3 Decoding performance . . . 21

5.4 Preliminary benchmarks: which runner wins where . . . 24

5.5 Conversion time . . . 25

5.6 Remaining measurements . . . 26

6 Limitations and future work **26**

7 Conclusion **27**

8 Appendix: build and conversion **27**

1 Introduction

LogGOPSim simulates parallel applications from GOAL schedules that are first converted into a binary trace. The original binary format fixes every node record at 39 bytes with a hard-coded set of fields. Producers that need additional information, such as which GPU performs an operation, therefore reuse existing fields: they pack it into the tag or into resource IDs. The file does not record this, so producer and simulator must agree on the convention implicitly, and a mismatch silently produces wrong results.

This project replaces that format with a **self-describing, extensible** one while keeping GOAL's operations and dependencies unchanged. The file header describes every field: its name, type, width and position. Producers can add named custom fields without changing the format, each simulator checks at load time that the file supplies what it needs, and fields are packed into only as many bits as their values require. Legacy files remain readable.

The project brief sets four goals. The table shows where each is addressed:

Goal	Question addressed	Section
Self-describing, extensible traces	How can the file describe compact fields and carry additional attributes?	3.1–3.4, 4
Load-time schema validation	How can a simulator check that the file supplies the values it requires?	3.5
Runtime specialization	How can decoding exploit a known schema instead of interpreting it per node?	3.6, 5.3
Benchmarking	Does the new format preserve correctness and improve size or speed?	5

Status: the format, converter, validating reader and LogGOPSim integration are implemented and tested, and ATLAHS's htsim simulator uses the same reader with a new GPU attribute (Section 4). Legacy files remain readable through the same reader, without format-specific code in either simulator (Section 3.5.2). Runtime specialization is implemented as five interchangeable plan runners: an optimized interpreter, two compiled decoders (system C compiler and LLVM ORC) and two template-based ones (Section 3.6). Preliminary measurements show decoding close to legacy speed and identify which runner is fastest in which scenario (Sections 5.3 and 5.4); the full performance evaluation is still outstanding.

2 Background

2.1 GOAL schedules

GOAL (Group Operation Assembly Language) describes the computation and communication of a parallel application as a schedule. It is an intermediate format: a simulator never sees the application itself, only a GOAL schedule describing what each process computes and sends. Schedules are rarely written by hand. Separate generator tools produce them, for example:

- **Schedgen** (and its Python rewrite Schedgen2), included with LogGOPSim, generates schedules for MPI collective operations and ML-training communication patterns, and converts MPI traces recorded with the liballprof tracing library.
- **ATLAHS** generates schedules from profiled AI training runs (NCCL communication) and from HPC application traces.

Because every generator emits the same text format, any of them can feed the same simulator. GOAL has three operations:

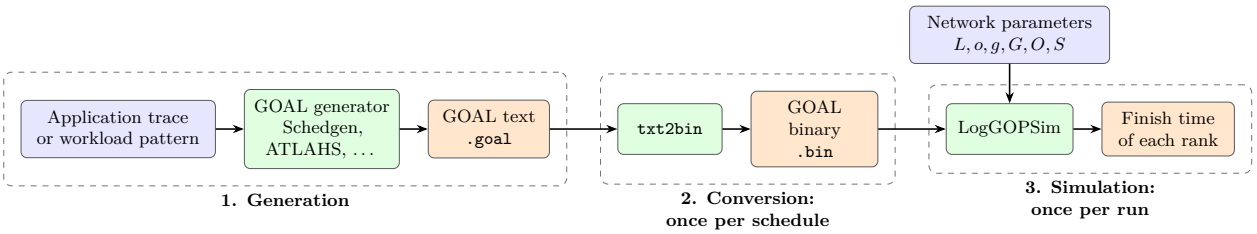
- `send`: send a number of bytes to another rank.
- `recv`: receive a number of bytes from another rank.
- `calc`: compute locally for a given amount of simulated time.

Operations belong to numbered **ranks**, usually application processes. Within a rank, operations form a dependency graph: `requires` waits for another operation to finish, `irequires` waits for it to start. Sends and receives in different ranks are connected by message matching on source and **tag**, an integer message label chosen by the application:

```
l1: calc 1000
l2: send 1024b to 1 tag 42
l2 requires l1
```

2.2 Workflow: generation, conversion and simulation

Getting from an application to simulated times takes three steps, each run as a separate program:



Any generator can produce GOAL; conversion runs once per schedule, and the binary is simulated as often as needed.

1. **Generation.** A generator produces the GOAL text from a recorded trace or a workload description, as described above. This step is outside LogGOPSim.
2. **Conversion.** `txt2bin` parses the text schedule, resolves labels such as `l1` to node indices, builds each rank's dependency graph and writes it as binary records. This happens once per schedule.
3. **Simulation.** LogGOPSim memory-maps the binary file and simulates it with a given set of network parameters. It does not parse text: it reads each node's record directly when it needs it. By default, simulation leaves the binary file unchanged so it can be reused.

The split between conversion and simulation pays off because parsing is done once while simulation is typically repeated, for example to sweep network parameters or compare network models. The binary file is the interface between the two steps, which is why its format is the subject of this project: Sections 3.1–3.4 cover the conversion side and Section 3.5 the simulation side.

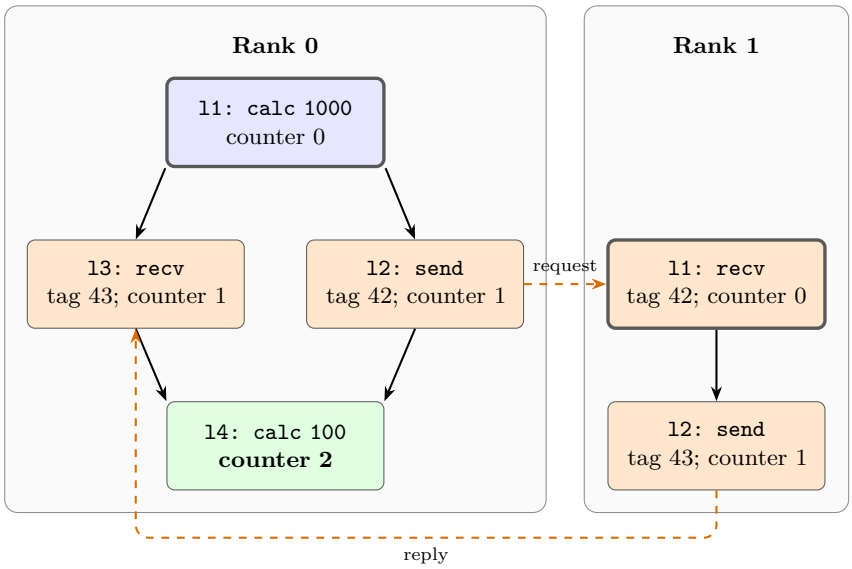
2.3 Simulation

LogGOPSim does not run the application. It walks the schedule's dependency graph and computes **when** each operation would start and finish, advancing a simulated clock. Each rank's operations are nodes in a graph. A solid edge is a local dependency: `requires` releases the dependent when the operation **completes**, `irequires` when it **starts**. Messages between ranks are not edges in the file; the simulator pairs each send with a matching receive at run time.

Every node carries a **dependency counter**: the number of its prerequisites that have not yet been satisfied. Nodes with a counter of zero at the start are the **roots**. In this example, rank 0 branches and joins again, while rank 1 receives a message and sends a reply:

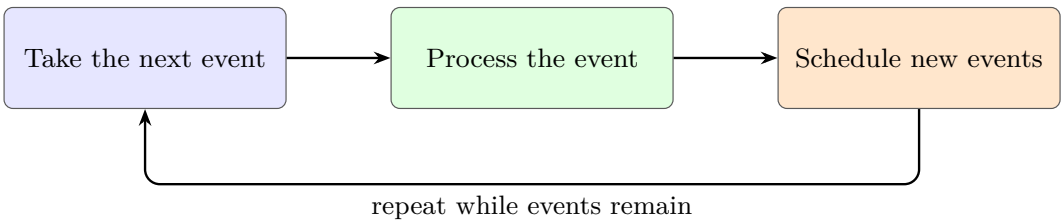
```
num_ranks 2
rank 0 {
  l1: calc 1000
  l2: send 1024b to 1 tag 42
  l3: recv 1024b from 1 tag 43
  l4: calc 100
  l2 requires l1
  l3 requires l1
  l4 requires l2
  l4 requires l3
}
rank 1 {
  l1: recv 1024b from 0 tag 42
  l2: send 1024b to 0 tag 43
  l2 requires l1
}
```

On rank 0, finishing `l1` makes both the send and the receive ready. Rank 1 receives the request before sending its reply. Rank 0's final calculation waits for both its send and its receive to finish:



Solid arrows: local completion dependencies. Dashed arrows: matching messages between ranks. Roots have a thick border.

The simulator processes events in time order, starting with the root operations:



Events are processed in simulated time order.

Processing an event updates the simulated time and operation state, including matching sends with receives. Satisfied dependencies reduce the waiting nodes' counters; reaching zero makes a node ready to run. The result is a predicted finish time for each rank.

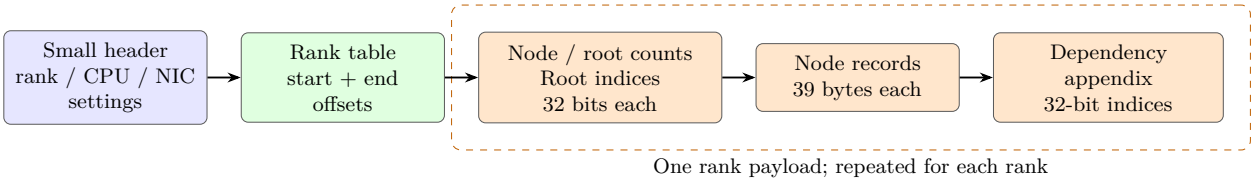
This determines what a binary trace must provide for every node:

- **Operation fields** used when the node is processed: type, peer, size, tag, CPU and NIC.
- **Graph fields** used to release dependents: the dependency counter and the lists of nodes released when a node starts and when it finishes.
- **The roots** of each rank, which seed the event queue.

Two properties matter for the format. First, a node is needed only when its counter reaches zero, so nodes can be read on demand instead of all at once. Second, the dependency counters are the only values that change during the run; everything else in a node record is read-only.

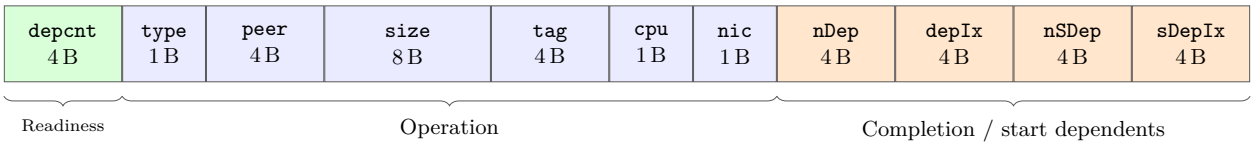
2.4 The legacy binary format

The legacy file starts with a small header containing the rank count and the largest CPU and NIC IDs. A rank table gives the start and end offsets of each rank's data. Within each rank, node and root counts and the root indices come before the node records and dependency lists:



Legacy file order: roots precede the fixed-size records and dependency appendix.

Every node record has the same eleven fields:



The legacy node record.

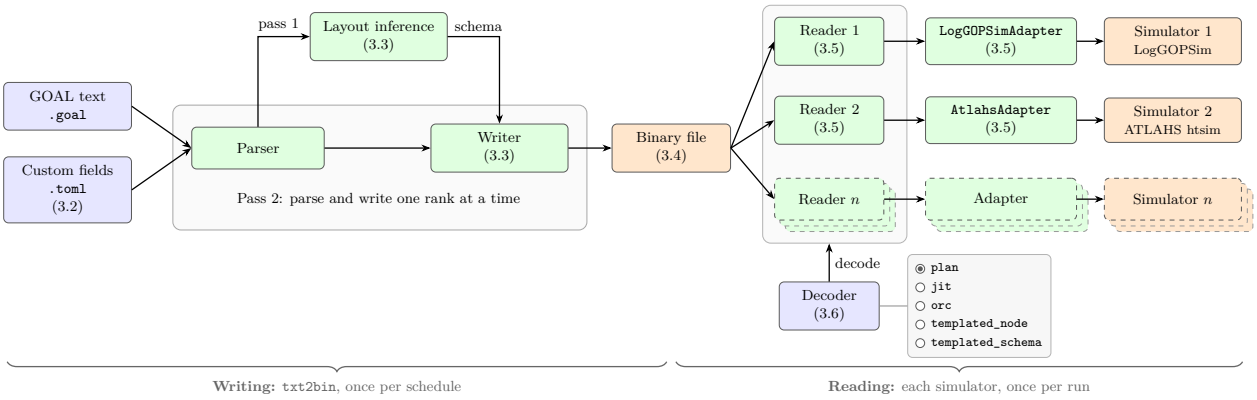
This layout has two problems. Every value occupies its full width regardless of the values that actually occur, and there is no place for additional information, so producers reuse existing fields. Section 4 shows a concrete case.

2.5 Related work

Source	Lesson used here
Avro	Store the writer's schema in the file and resolve the reader's requirements against it before decoding.
Arrow	Separate storage types from meaning; allow named extension fields.
OTF2	Define names and types once, separately from event values.
Chakra	Core graph fields plus typed custom attributes.
Cap'n Proto / FlatBuffers	Specify encoding, alignment and evolution rules explicitly.
HyPer / Excalibur	Generate specialized code for the hot path; keep a generic path alongside it and account for compile latency.

3 Design and implementation

The new design follows a trace from text to simulation. The numbers identify the relevant subsections:



Pass 1 determines the schema for the whole input. Pass 2 alternates parsing and writing for each rank. Numbers refer to subsections.

The code separates conversion from simulation: `txt2bin.cpp` coordinates parsing, layout inference and binary writing. Each simulator is a separate executable with its own adapter and reader, which decodes only the fields its node type needs, using one of the plan runners (Section 3.6).

The main source folders, relative to `src/LogGOPSim/`, follow these responsibilities:

Folder	Responsibility
`goal_parsing/`	Parse GOAL text, load custom-field declarations and collect ranges and counts for layout inference.

Folder	Responsibility
`binary_writing/`	Pack records and ranks, and write legacy or self-describing binary files.
`trace_format/`	Define shared field types, encodings, record layouts, headers and rank tables.
`binary_reading/`	Map binary files, detect their format, validate them, decode selected fields from either format and update dependency counters.
`simulators/`	Provide the shared adapter interface and graph-node base, with simulator implementations and adapters under `loggopsim/` and `atlahs/`.

3.1 Standard and custom fields

Not every simulator needs every standard field, so we considered keeping only what is needed to traverse the graph (dependency counts, dependency lists and roots) and making all other fields custom. We kept `type`, `peer`, `size` and `tag` as standard fields because they are part of GOAL itself: every operation is a send, receive or calculation with a peer, size and tag, so every GOAL simulator needs them. `cpu` and `nic` are also part of GOAL, where an operation may omit them and they default to zero, so they stay standard fields as well. They describe LogGOPSim's resource model; ATLAHS, for example, selects GPUs and streams through custom fields instead and simply does not read them. A file may contain fields that a given simulator does not need, since each simulator reads only the fields it binds, and a field with the same value on every node takes no bits per node. Custom fields are therefore reserved for information that GOAL does not define.

3.2 Custom fields

Custom fields are declared in `config/custom_fields.toml`, or a TOML file selected with `txt2bin --custom-fields <file>`, each with a name and an integer type:

```
[[fields]]
name = "gpu_id"
type = "unsigned"

[[fields]]
name = "priority"
type = "signed"
```

Producers append `name value` pairs to every operation, in any order:

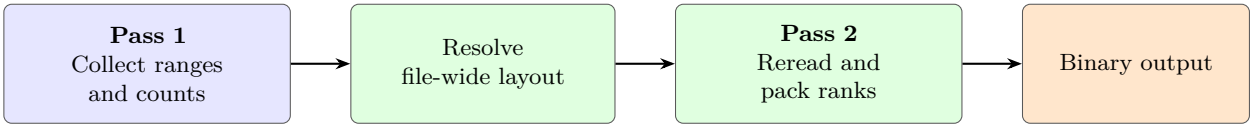
```
l1: send 1024b to 1 tag 42 gpu_id 3 priority -2
l2: recv 1024b from 1 tag 42 priority 0 gpu_id 3
l3: calc 1000 gpu_id 3 priority 0
```

Invalid or duplicate declarations are rejected, and every declared field must appear exactly once on every operation; missing or invalid values are rejected during conversion. From then on, custom fields are treated exactly like standard fields: their ranges are collected, their widths inferred and their values packed into the same record. Legacy output cannot store them.

Custom fields currently support signed and unsigned integers. Types are defined in one module shared by parsing, statistics and the layout builder, so adding a type means extending that module, the encoding and the parser; float and string types are not implemented.

3.3 Inferring the layout

A compact layout must be chosen before the first record is written: if tags `0..7` fit in three bits but a later rank uses tag `1000`, records already written cannot be widened. The converter therefore reads the input twice:



Infer the layout first, then use it to pack every rank.

The first pass keeps only file-wide statistics: each field's minimum and maximum, whether wildcards occur, dependency counts and the largest rank's node count. The second pass parses each rank again and writes its packed records, dependencies and roots using the final layout.

GOAL input is grouped by rank. Each pass builds one rank's graph, collects its statistics or writes its binary data, then frees the graph's memory before parsing the next rank. Graphs for all ranks therefore do not need to fit in memory together, and no temporary file is needed.

This keeps graph memory bounded by the largest rank, but repeats parsing and graph construction. A single rank with a large value range also determines that field's width for the whole file. The alternatives below include ways to avoid the second parse and a way to reduce the work done in each pass.

Approach	**How it works**	**Main tradeoff**
Two passes (current)	Collect ranges across all ranks, choose one layout, then parse again and write each rank.	Holds one graph at a time and needs no temporary file, but repeats parsing and graph construction.
Lighter passes (proposed)	Reuse the same lexer. Pass 1 collects field ranges and per-node dependency counts without building the full graph. Pass 2 builds and packs each rank using the resolved layout, without repeating range collection.	Still reads the text twice, but avoids unnecessary graph allocations in pass 1. Label tracking and dependency counts remain necessary; validation must stay complete. The speedup needs measurement.
Original fixed widths	Parse and write each rank immediately using predefined field widths.	Needs only one pass, but reserves space even for small values. Values outside the fixed range must be rejected.
Keep all graphs	Parse once and retain every rank graph. Choose the layout after the last rank, then pack the retained graphs.	Avoids reparsing and temporary I/O, but holds the entire expanded schedule in memory, including graph and allocation overhead.
Temporary binary file	Parse once, collect ranges and store values and dependencies temporarily. Reread them to pack the final file.	Keeps graph memory bounded and avoids a second text parse, but adds temporary storage and a full write/read. Speed depends on storage and caching.
Per-rank layouts (proposed)	Parse a rank, infer its local layout, write it and free its graph. The rank table locates the data; local metadata describes its layout.	One pass and potentially narrower fields, but repeated metadata and decoder setup can outweigh savings for small ranks. Requires a format change.

The temporary-file experiment was removed. On a synthetic 2.1-million-node trace it reduced conversion time from 2.32 s to 1.71 s (1.36×), using 193 MB of temporary data. This used a warm file cache without measuring peak memory; it is not reproducible with the current code and does not establish a general speedup.

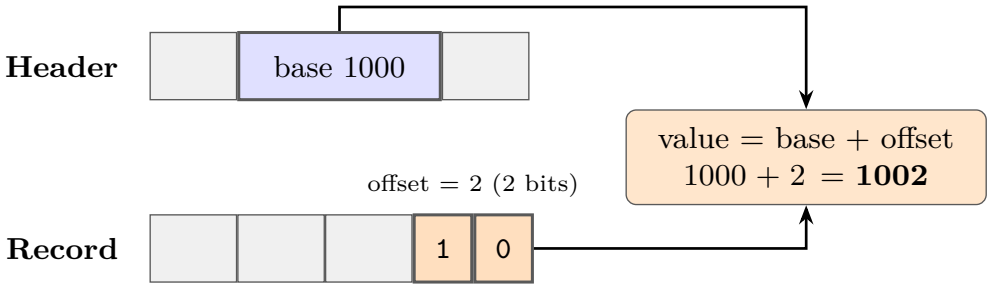
Alexander Falter is currently implementing a lighter first pass on the GPU. Keeping all graphs is infeasible for large schedules, but the options need not exclude each other: the converter could choose by input size, keeping all graphs in memory when the schedule is small enough and otherwise falling back to a temporary file or two passes.

3.3.1 Field encoding

Choosing an encoding. Sizing each field to its largest value already avoids the legacy format's fixed widths, but a field still costs the bits of its largest value, however little it varies. An encoding stores something smaller than the value itself. If a field is 1024 on every node, the plain value needs 11 bits per node; with offset encoding, 1024 is stored once in the header and each node needs 0 bits. If a field alternates between 1 and 1,000,000, the plain value needs 20 bits, and offset encoding does not help, because offsets still reach 999,999; with dictionary encoding, the header stores the list [1, 1000000] once and each node stores only its index, 1 bit. Offset encoding therefore suits values that lie close together, and dictionary encoding few values far apart. On a Schedgen binomial-tree reduction over 262,144 ranks, offset encoding alone shrinks the record from 39 to 5 bytes (Section 5.2). Combining both, by choosing the cheaper encoding for each field, would probably work best.

Each field is stored as an offset from the smallest observed value. A field with minimum `m` and maximum `M` uses `bit_length(M − m)` bits, and the header stores the base `m`; a constant field therefore uses no bits at all.

The reader reconstructs each value as base plus offset:



Offset encoding: the header stores the base once; each record stores only the offset.

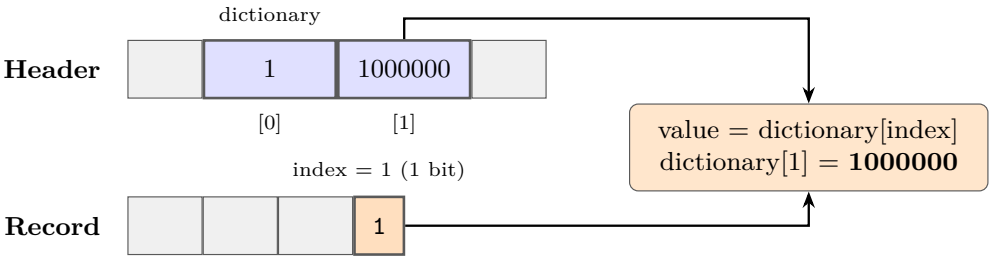
If a field uses the wildcard ANY (a receive from any source or with any tag), one extra code above the maximum represents it. For example:

Values	Base	Stored codes and width
1000, 1001	1000	0, 1: one bit instead of eleven
only 1024	1024	none: zero bits, decoded from the base
0–7 plus ANY	0	0–7 and 8 for ANY: four bits

The same rule applies to the graph fields. The dependency counter's range must start at zero, because counters are decremented to zero during simulation. Dependency and root indices refer to nodes within a rank, so their width follows from the largest rank's node count. The four dependency-list fields occupy 16 of the 39 legacy bytes, so packing them matters as much as packing the operation fields.

`txt2bin --report-schema` prints the inferred layout, including every field's range, base, width and position.

Dictionary encoding (proposed). Offset encoding can waste bits when a field has only a few distinct values spread over a wide range. For example, `{1, 1000000}` requires 20 bits with offsets. Instead, the converter could use a hash map to assign each distinct value a compact code: `1` → `0` and `1000000` → `1`. Each record would then need only one bit for this field. The header would store the reverse dictionary `[1, 1000000]`, allowing the reader to recover the original value by indexing it with the stored code:



Dictionary encoding (proposed): the header stores the distinct values; each record stores only an index.

The codes are unique identifiers, not hash values, so hash collisions do not lose information.

This helps when a small set of values occurs repeatedly. The cost is keeping the distinct values during analysis, storing their dictionary in the header, and performing a lookup during encoding and decoding. If most values are distinct, this overhead can outweigh the savings; the converter could compare the total size with offset encoding before choosing. Supporting this requires an encoding-kind marker and dictionary support in the writer and reader. Mutable dependency counters would need special handling because decrementing a code does not decrement its original value.

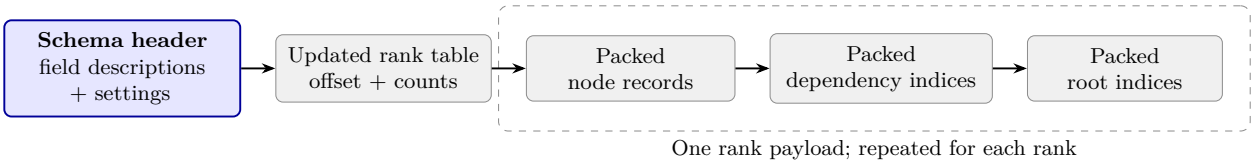
Combining both encodings (proposed). Neither encoding is best for every field, so the converter would choose per field:

Field values	Offset (per node)	Dictionary (per node, plus header)	Choice
0–262,143 (dense range)	18 bits	18 bits, plus 262,144 values in the header	offset
always 1024	0 bits	0 bits, plus 1 value in the header	offset
8 B, 1 MiB, 64 MiB (few, far apart)	26 bits	2 bits, plus 3 values in the header	dictionary
{1, 1000000}	20 bits	1 bit, plus 2 values in the header	dictionary

The first conversion pass already collects each field's range; it would also count distinct values up to a limit. For each field, the converter compares the record bits times the node count plus the dictionary's size in the header, and records the chosen encoding in the field descriptor. Dependency counters always use offsets. Each field then costs at most what either encoding alone would, which matters most for fields such as message sizes, whose few distinct values are often far apart.

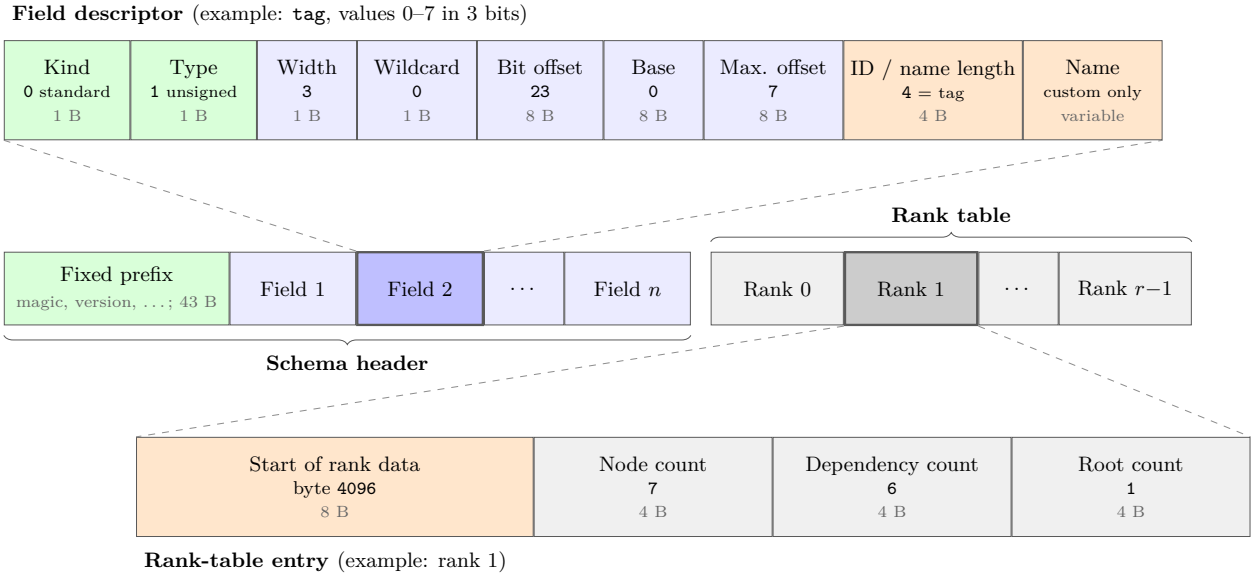
3.4 The file format

The new format keeps the same basic components: a rank table, node records, dependency lists and root indices. These still serve the same purposes as in the legacy format; the records and indices now use inferred bit widths instead of fixed widths, changing their storage sizes. The main addition is the schema in the header, which describes this encoding. The existing components are shown in grey:



Blue: the schema added to the header. Grey: existing components, with records and indices stored at inferred widths.

The rank counts move into the rank table, and root indices follow the dependency appendix. Records and indices use inferred bit widths. The schema header makes these encodings discoverable by the reader:



Header and rank table in file order, with one field descriptor and one rank-table entry enlarged; the values are an example and cell widths are illustrative. The fixed prefix holds the magic number, version, header and record sizes, field and rank counts, index width and table offset.

All node records in a file share one layout, so node `i` of a rank is found directly at `rank_start + i` × `record_size`, without scanning. Dependency and root entries are packed node indices using the file-wide index width. Multi-byte integers are little-endian and bits are numbered from the least significant bit of the first byte, so files are portable between machines.

The magic number distinguishes the new format from legacy files, which keep their own magic and remain readable (Section 3.5.2). Readers reject unknown versions, unknown type codes, truncated headers, overlapping fields and table entries pointing outside the file.

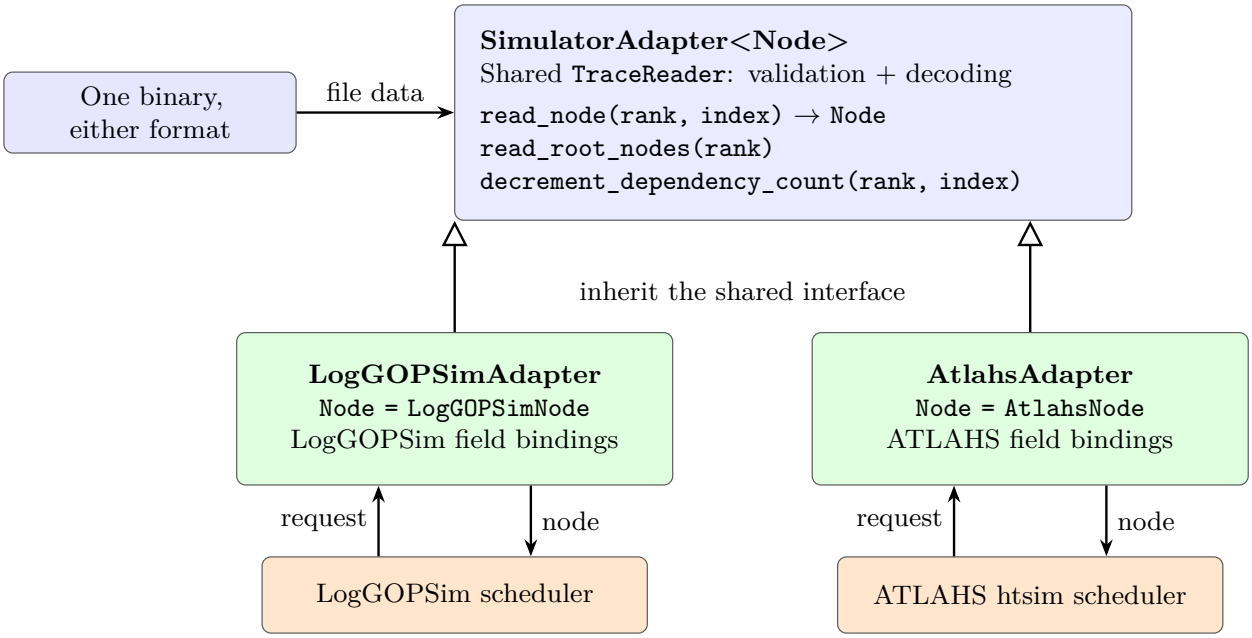
Legacy output remains the default for existing workflows. `txt2bin --format self-describing` selects the new format, which also supports custom fields; legacy output warns that it omits them.

3.5 Reading through simulator adapters

Reading is configured for the simulator that will consume the nodes. The binary may contain more fields than that simulator needs: its adapter selects the fields to decode and specifies where each value goes in the simulator's node struct. Shared graph fields, such as the dependency counter, are filled automatically; other unbound fields are skipped. The same binary can therefore be read into different node types without converting it again.

3.5.1 Connection and node reads

Each simulator creates its own adapter object. The adapter provides the connection between the stored records and the simulator's input interface:



Both adapters inherit the same interface, specialized for their node type. Each simulator owns a separate adapter and reader instance.

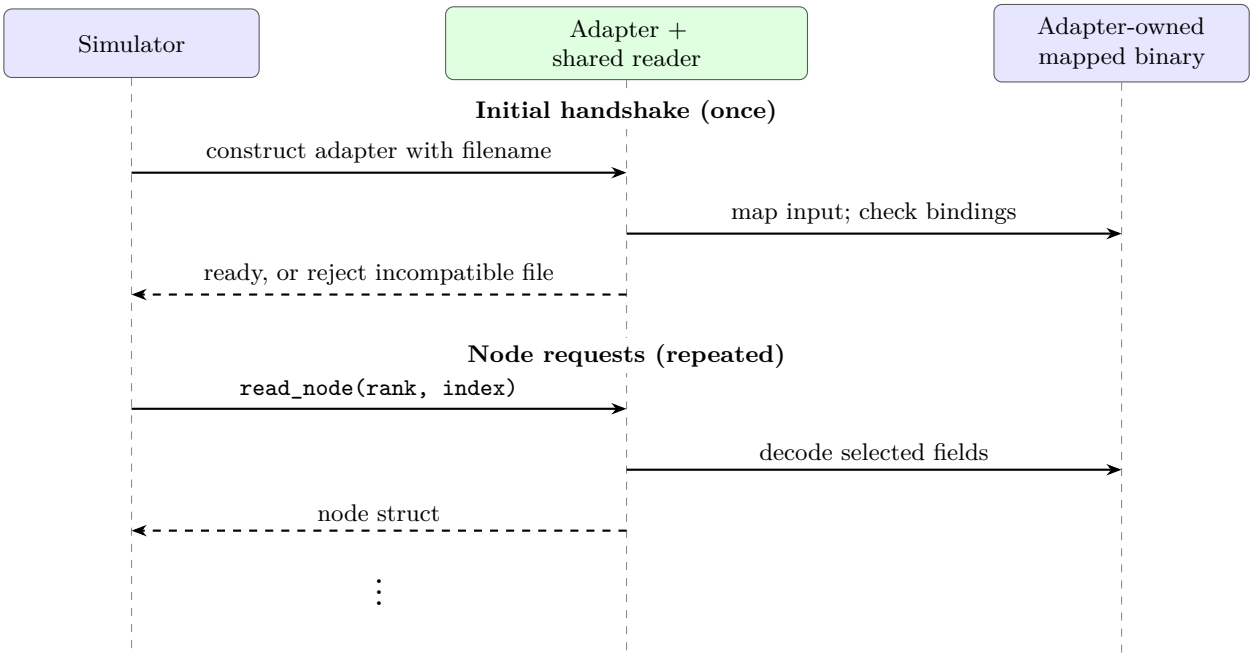
The node type tells the adapter what to return; it is not a buffer containing every node. Both adapters use the same reader implementation. For file input, each adapter owns its reader and mapping, keeping the bytes alive until the reader is destroyed. The simulators are independent programs and can read a compatible binary concurrently using separate private mappings. Both must support its operations and message-matching convention; the explicit ATLAHS convention in Section 4 currently requires its backend.

All graph simulators' node types inherit a common base that the reader fills automatically: the node's index, its remaining dependency count, and the nodes it releases when it starts and when it completes. The adapter adds only the members its model uses, so supporting a new simulator means declaring its node type and bindings and making its input loader call the reader.

LogGOPSim itself was restructured into the first adapter. Its simulator code moved into its own directory; at startup its adapter opens the file, and the shared reader selects the legacy or self-describing implementation. Event timing, message matching and dependency processing are shared by both formats, so only storage access differs. Along the way, legacy reads were made safe for unaligned data, and empty schedules are now supported.

The procedure has four steps:

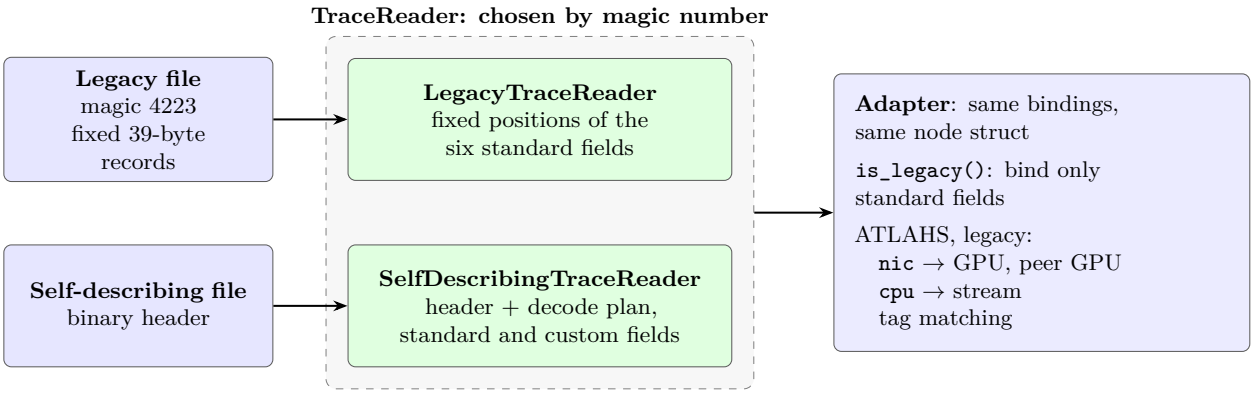
1. **Define the connection.** The adapter names its node type and binds file fields to that type's members. The member's C++ type tells the reader which values it can accept.
2. **Prepare once.** The simulator supplies the filename to its adapter, which opens and owns the mapping and constructs the reader. The shared reader checks the schema against the bindings and resolves the selected fields' positions and encodings. Missing fields, incompatible types, values outside the destination type's range and unsupported wildcard representations are rejected before simulation starts.
3. **Request a node.** The scheduler calls `adapter.read_root_nodes(rank)` to get initially ready nodes, or `adapter.read_node(rank, index)` to get a particular node. The reader locates that record and decodes the bound fields directly into the chosen node type. For ATLAHS the result is an `AtlahsNode`; for LogGOPSim it is a `LogGOPSimNode`.
4. **Simulate and continue.** The simulator processes the returned node using its own timing and communication model. As prerequisites are satisfied, it updates dependency counters through the adapter and requests newly ready nodes. The scheduler decides which node to request; there is no sequential `get_next_node()` operation.



Construct once, then read nodes on demand.

3.5.2 Backward compatibility

Existing legacy traces must keep working without conversion, and the simulators should not need a second code path for them. The adapters therefore do not read files themselves: they hold a `TraceReader`, which looks at the file's magic number and creates one of two readers. Both accept the adapter's field bindings and fill the same node struct.



The format is detected once; the adapter's bindings and node type are the same for both.

- **Self-describing files** are read through the header and a decode plan (Section 3.6); any standard or custom field can be bound.
- **Legacy files** are read by a small reader that knows the fixed position and width of each standard field in the 39-byte record: type, peer, size, tag, CPU and NIC, plus the dependency count and lists. Each binding becomes a copy from that position into the bound member, so one field may be bound to several members. The member must hold every value of the stored width. Peer and tag store "any" as all ones, which becomes the binding's wildcard value. A binding to a custom field is rejected, because the legacy record cannot contain one.

An adapter can ask `is_legacy()` to bind only the fields that exist. LogGOPSim binds the same six standard fields for both formats. ATLAHS binds its eight custom fields only for self-describing files. For legacy files it keeps the upstream convention: the NIC field is bound to both `Gpu` and `PeerGpu`, the CPU field to `Stream`, and messages are matched by tag (Section 4). Neither simulator's scheduler contains format-specific code.

The legacy reader validates each rank's bounds, its roots and its dependency indices when the file is opened, as the self-describing reader does. Files invalidated by an earlier `--save-mem` run are rejected with a clear error. The converter still writes the legacy format by default. On the benchmark traces, LogGOPSim's output is identical to that of the original LogGOPSim for both formats (Section 5.3.2).

3.5.3 Validation before and during simulation

The schema check compares the destination member's type with the **decoded value range**. The stored bits represent an offset, and decoding reconstructs the original value as `base + offset`. For example, a 2-bit field stores offsets `0..3`; with a base of `1000`, it represents values `1000..1003`. Copying such a value into an 8-bit unsigned member would silently truncate it, for example 1002 to 234. The stored code fits in two bits, so a check on the stored width would accept this binding; the reader therefore checks the decoded range, base plus maximum offset, against the member type and rejects the file before any node is read.

Validation is divided between setup and individual reads:

- **At setup:** check the schema and bindings, file sections and root lists. Roots must be ordered, distinct, within their rank and initially have a zero dependency counter.
- **When reading a node:** check record bounds, dependency indices, operation codes, peer ranks and invalid send wildcards. A simulator can add checks for its own model; ATLAHS, for example, rejects messages larger than htsim supports.

3.5.4 Accessing mapped data

The reader accesses the binary through a memory mapping. The scheduler starts from each rank's roots and requests dependent nodes as their counters reach zero, so the reader decodes nodes on demand.

Dependency lists remain views into the mapped bytes rather than being copied into a new vector for every node. Graph setup validates the immutable dependency indices once; each node read still checks its list bounds, without scanning the indices again. Field-only consumers do not scan dependency arrays. The mapping must stay alive while the lists are used. A consumer that needs a list after closing the mapping must copy its indices first. Analysis tools can bind selected fields into a small struct without inheriting the graph-node base.

3.5.5 Dependency counters and file reuse

Dependency counters change during simulation; the other stored node values stay unchanged. The reader currently decrements counters in the mapped bytes. With the default private mapping (`MAP_PRIVATE`), the operating system gives a process its own copy of a page when it modifies it. The original file stays unchanged, and each simulation uses a fresh mapping, allowing independent runs to reuse the same binary, including concurrently.

LogGOPSim's `--save-mem` instead uses a shared mapping: counter changes reach the file on disk, which is marked unusable for later runs. This mode is destructive and must not be used on an input shared by several simulations. ATLAHS accepts it only for legacy traces, as upstream did; for the new format it always uses a private mapping, because packed counters share bytes with other fields.

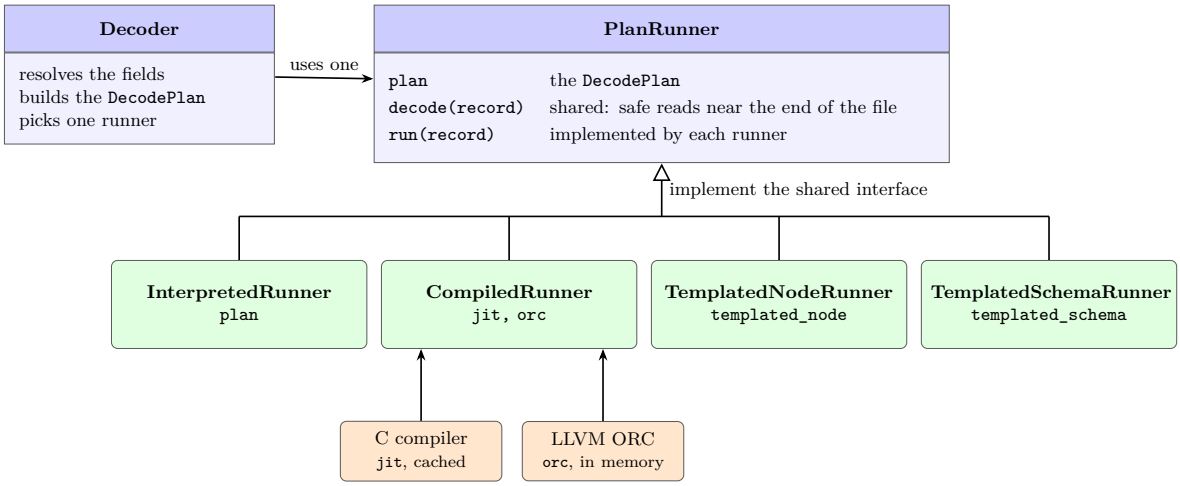
Private mappings copy whole pages even when only a counter changes. Keeping the trace read-only and storing counters separately for each run is a possible improvement; its memory and access costs still need measurement.

3.6 Runtime specialization

Runtime specialization replaces the shared reader's generic field-decoding loop with code prepared for the opened file and the simulator's bindings. The adapter still exposes the same node-reading interface, and the simulator requests nodes in the same way.

3.6.1 Where it fits

The file schema and adapter bindings are already resolved at setup. Together, they determine which bits to read, how to reconstruct each value and which node member receives it. The reader turns this information into a specialized decoder once, then uses it for every node read:



All runners share boundary handling; compiled runners execute C- or LLVM-generated code.

The decoders are alternatives behind the same reader interface. Record bounds checks, dependency views and counter updates stay in shared code. Each simulator uses a decoder prepared for its own bindings, even when both read the same binary.

3.6.2 Why it could be faster

The generic decoder visits the selected fields for every node, reading their positions, widths and bases from prepared metadata, passing values through `std::variant` and invoking an assignment function per field. A specialized decoder puts these choices directly into its instructions. For a field stored at bit position 13 with width 2 and base 0, for example, it extracts those known bits and stores the result directly into `node.Gpu`.

Knowing these constants lets the compiler simplify shifts and masks, remove the per-field loop and indirect assignments, and combine nearby loads. Field selection already avoids decoding unwanted values;

specialization reduces the cost of decoding the selected ones. It does not speed up message matching or network simulation.

3.6.3 Choosing an approach

Approach	How it works	Tradeoff
Template instantiation	Precompile decoders for known layouts and pick one at run time	No compiler needed, but covers only layouts known in advance
Generate C, compile, load	Emit source for this schema, call the system compiler, load the shared library	Fully general and easy to inspect; compile latency, mitigated by a cache
In-process JIT	Build machine code in memory, e.g. with LLVM ORC or asmjit	No external compiler, but a large dependency or per-architecture code
Optimized interpreter	Precompute shifts and masks once and run them in a tight loop	Always works; some overhead remains

We implemented all four approaches, the in-process JIT with LLVM ORC. All are built from the same *decode plan*, so the interpreter also serves as the fallback when compilation is unavailable.

3.6.4 Decode plan and plan runners

When the reader is constructed, it builds a decode plan with one step per selected field: where the field's bits are (first byte, shift, mask, base, maximum offset) and where its value goes (output struct, member offset and size, value for ANY). The plan is only data. The `Decoder` resolves and validates the requested fields, builds the plan and selects a runner. The shared `PlanRunner::decode` uses that runner's fast `run` when its wide loads are safe, or bounded reads near the end of the mapping. Invalid records are rejected directly. The environment variable `LOGGOPSIM_DECODER` selects the runner; the shared interface and implementations are shown above.

The runners are:

- `plan` (default, `InterpretedRunner`): runs the steps in a loop, without `std::variant` or `std::function`.
- `jit` (`CompiledRunner` with the system C compiler): turns the steps into a C function, compiles it with `cc -O2 -shared -fPIC` and loads it with `dlopen`.
- `orc` (`CompiledRunner` with LLVM ORC): builds the same function as LLVM IR and compiles it in-process, without a compiler process, file or cache. The LLVM code is a plugin, opened only when `orc` is selected, so other runs do not load LLVM: linking LLVM into the simulator directly added 7–11 ms to every run. Builds without LLVM fall back to the interpreter. Within one process, readers of the same plan reuse the compiled function. It decodes as fast as `jit` (about 66 ns per node in `decode_bench` on a 524,286-node trace) and spends about 3 ms more on setup than a cached `jit`, because it compiles on every run.
- `templated_node` (`TemplatedNodeRunner`): a template instantiated for each simulator's node type, which fixes the number of steps and each member's size and output struct. The steps are unrolled and every store has a fixed size; bit positions, ranges and member offsets still come from the plan, so one instantiation serves every trace a simulator reads. It needs no compiler at run time.
- `templated_schema` (`TemplatedSchemaRunner`): a template that takes every step as a template argument, so all positions, masks and ranges are constants, as in the generated C. It only matches traces with exactly the layout it was instantiated for; selecting it on another trace reports the missing match once and falls back to the interpreted runner.

Runner registry. The runners are listed in one place, `RunnerRegistry.cpp`. Each entry gives the runner's name, a description, whether it handles any plan, whether it is available in this build (for example, whether a C compiler or the LLVM plugin is present) and a function that creates it for a plan. `plan` is the default and the fallback. The decoder looks up `LOGGOPSIM_DECODER` in the registry and rejects unknown names with the list of valid ones; if the runner cannot handle the trace, it says so once and uses `plan`. The unit tests and the benchmarks take their runners from the same list, so a new runner is tested and measured without further changes. Each runner lives in its own folder with its own build file; adding one means adding the folder, one `add_subdirectory` line and its registry entry, and removing one is the reverse.

All five runners give identical simulation results. Replaying the 524,286-node trace (`decode_bench`, 7 interleaved rounds of 20 repetitions, median per repetition):

Runner	ns/node	Setup (ms)
`plan`	96.7	18.4
`templated_node`	84.3	16.1
`jit` (cached)	66.5	13.1
`orc`	66.5	15.9
`templated_schema`	66.2	12.9

`templated_schema` is as fast as the compiled runners: with all constants known, the compiler produces the same code. It is the fastest option without a run-time compiler, but only for traces whose layout was compiled in, and layouts change with each trace's value ranges. We therefore use it as a reference, showing what compiled code costs without compilation, rather than as a runner to choose for new traces. `templated_node` helps every trace of a known node type and is 13% faster than `plan`; it stays slower than the compiled runners because each step still loads its shift, mask and range from memory.

The generated function contains every offset, shift, mask and range as a constant, with no loop or dispatch; a constant (0-bit) field becomes a single store. An abridged excerpt for a broadcast trace:

```
int decode(const u8* r, u8* const* t) {
  u64 w, c;
  memcpy(&w, r + 0x0, 8);          // type: 2 bits at bit 1, base 1
  c = (w >> 0x1) & 0x3;
  if (c > 0x2) return 0;
  c += 0x1;
  store8(t[0] + 0x38, c);
  memcpy(&w, r + 0x0, 8);          // peer: 16 bits at bit 3
  c = (w >> 0x3) & 0xffff;
  store32(t[0] + 0x3c, c);
  store64(t[0] + 0x40, 0x400);     // size: 0 bits, constant 1024
  /* ... further fields ... */
  return 1;
}
```

The source contains only numbers taken from the header, never strings from the trace, so a trace cannot inject code.

Fallbacks. If a fast path rejects a record (a value outside its range, a send to ANY, or a record too close to the end of the file for a 64-bit load), the base class decodes it again exactly and reports the error, so error messages are unchanged. If compilation or loading fails, the reader uses the interpreted decoder. The dependency counter has a matching fast path for decrements.

Cache. Compiled libraries are cached under `~/.cache/loggopsim-jit` (configurable with `LOGGOPSIM_JIT_CACHE`), keyed by a hash of the generated source, compiler and flags. The source encodes the layout, bindings and destination offsets, so the key covers them. Concurrent builds write under a temporary name and rename. The key does not include the compiler version or target architecture; on a home directory shared between different machines, loading fails and the reader falls back to the plan.

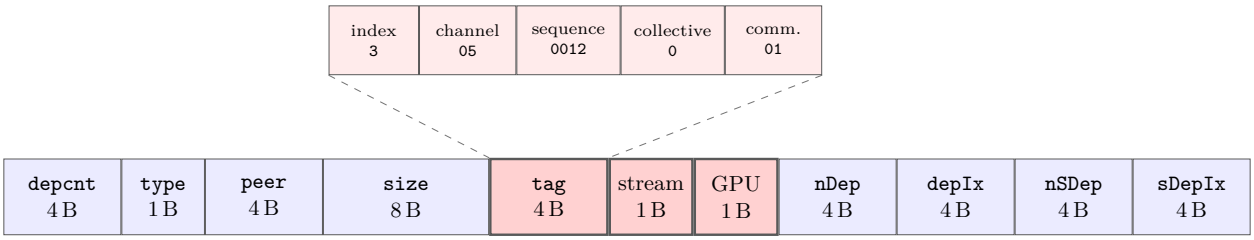
Section 5.3 evaluates the runners against the original decoder, and Section 5.4 compares them across scenarios.

4 Case study: explicit ATLAHS metadata

ATLAHS provides workload generators for AI and HPC applications and the htsim packet simulator. We use it as a second consumer of the new format.

4.1 Original encoding in standard fields

The legacy record has no GPU field. ATLAHS's NCCL generator models several GPUs per host, so it reuses three existing fields (red):



How ATLAHS fills the legacy record; nothing in the file records this.

The file cannot say what these values mean: a `nic` value might be a NIC or a GPU, and the tag's components exist only in the generator's code. htsim therefore guesses the convention from the largest CPU and NIC IDs in the header. A trace that does not fit the guess is simulated with the wrong GPU mapping and no error, and a trace without GPU information cannot be rejected.

4.2 Explicit GPU, stream and message fields

The generator now emits local and peer GPU IDs, the stream and all five message-identity components as separate attributes. The standard CPU, NIC and tag remain zero:

```
before: l2: send 1024b to 1 tag 3050012001 cpu 0 nic 2
after:  l2: send 1024b to 1 tag 0 cpu 0 nic 0 stream_id 0 gpu_id 2 peer_gpu_id 2 chunk_id 3 channel_id 5
     sequence_id 12 message_kind 0 communicator_id 1
```

The eight attributes need no change to the file format, converter or shared reader. They follow the same custom-field path:

1. Declare. The TOML file declares all eight attributes as unsigned integers:

```
[[fields]]
name = "gpu_id"
type = "unsigned"

[[fields]]
name = "communicator_id"
type = "unsigned"

[[fields]]
name = "channel_id"
type = "unsigned"

[[fields]]
name = "sequence_id"
type = "unsigned"

[[fields]]
name = "message_kind"
type = "unsigned"

[[fields]]
name = "chunk_id"
type = "unsigned"

[[fields]]
name = "stream_id"
type = "unsigned"

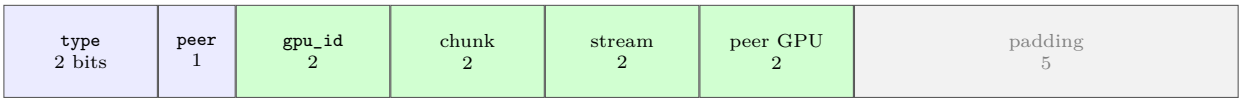
[[fields]]
name = "peer_gpu_id"
type = "unsigned"
```

2. Convert. `txt2bin` collects each field's range and chooses its width, as for standard fields.

3. Describe. The header stores each field's name, type, base, width and position.

For the four-GPU example trace, conversion produces this packed record layout:

16-bit record: standard fields (blue), custom fields (green)



0 bits, constant and read from the header: `depcnt`, `size`, `tag`, `cpu`, `nic`, the four dependency-list fields, communicator, channel, sequence and kind.

Four-GPU example: 2 bytes per node instead of 39, including all eight custom fields.

4. Bind. The ATLAHS adapter supplies this node type. It inherits the common graph fields and adds the operation fields and explicit metadata its scheduler needs:

```
struct AtlahsNode : BaseDeserializedGraphNode {
    std::uint8_t Type = 0;
    std::uint32_t Peer = 0;
    std::uint64_t Size = 0;
    std::uint32_t Tag = 0;
    std::uint8_t Proc = 0;
    std::uint8_t Nic = 0;
    std::uint32_t Gpu = 0;
    std::uint32_t PeerGpu = 0;
    std::uint32_t Stream = 0;
    std::uint64_t Communicator = 0;
    std::uint64_t Channel = 0;
    std::uint64_t Sequence = 0;
    std::uint64_t MessageKind = 0;
    std::uint64_t Chunk = 0;
    std::uint64_t start_time = 0;
};
```

For example, this binding maps the declared `gpu_id` field to `Gpu` in the returned node:

```
bind_field(std::string{"gpu_id"}, &AtlahsNode::Gpu)
```

The other seven attributes bind to `PeerGpu`, `Stream`, `Communicator`, `Channel`, `Sequence`, `MessageKind` and `Chunk` in the same way. Missing or incompatible fields are rejected at setup. `start_time` is runtime state maintained by the scheduler, rather than a stored field.

5. Use. `Gpu` selects the local GPU and `PeerGpu` the remote GPU; on a send these are the source and destination, respectively. `Stream` selects the execution stream. Receive matching compares the source and the `(Communicator, Channel, Sequence, MessageKind, Chunk)` tuple, and checks that both GPU endpoints agree with the receive.

With the new format htsim no longer guesses: the GPU count is the field's maximum plus one, read from the header. Legacy ATLAHS traces still work: for them the adapter binds the NIC to `Gpu` and `PeerGpu` and keeps the upstream mapping.

4.3 Integration

Declaring and binding the eight fields makes them available to ATLAHS, but its existing code also has to produce and use them. We changed three points along the path from workload generation to simulation.

4.3.1 Generator: emit the original values

Previously, the NCCL generator encoded the GPU in the NIC field and combined message-identity components into a decimal tag. We added a new option, `--explicit-fields`, to the generator. With it, every operation carries eight additional named attributes instead of packed numbers: the GPU it runs on, the GPU of its communication partner, its execution stream, and the five components of its message identity. To find the partner's GPU, the generator looks up which GPU the peer rank uses in that communicator and converts it to a dense host-local index (0, 1, 2, ...); this works for point-to-point, ring and tree communication. This preserves their values before they enter the converter, avoiding the ambiguity of reconstructing them from overloaded fields. Calculations and fences supply zero-valued identity attributes

because they do not participate in message matching. Generation without the option keeps the original output convention.

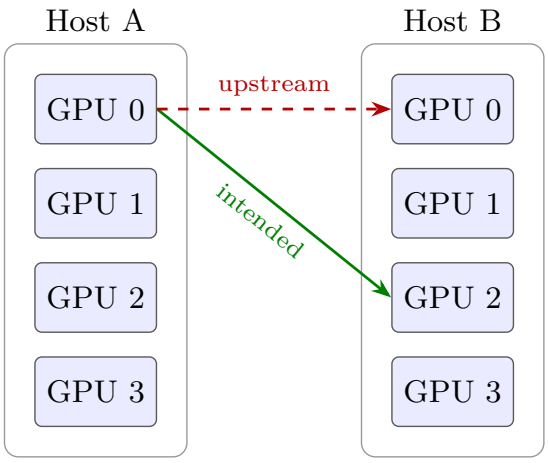
4.3.2 Input loader: construct the scheduler's nodes

The existing ATLAHS loader read legacy records using fixed byte offsets. It now constructs an `AtlahsAdapter` that owns the mapping and reads either format. When the scheduler needs a node, the loader obtains an `AtlahsNode` through that adapter. The shared reader handles field positions and decoding, while the adapter supplies the bindings and ATLAHS-specific checks. The rank scheduler reads only the dependency list released by each event. A single release helper updates counters and carries the latest prerequisite completion time into newly ready nodes. The old converter's graph writer is kept separately from this runtime loader.

This is where the generic format connects to the existing simulator: the scheduler receives the operation fields it already uses, together with explicit GPU, stream and message identity. Legacy files use the same adapter, which binds only the standard fields and binds the NIC to both GPU members to preserve their GPU mapping (Section 3.5.2).

4.3.3 Scheduler: use the explicit metadata

Upstream used the sender's GPU index at both network endpoints: a send from A/GPU 0 to a receive on B/GPU 2 still travelled to B/GPU 0.

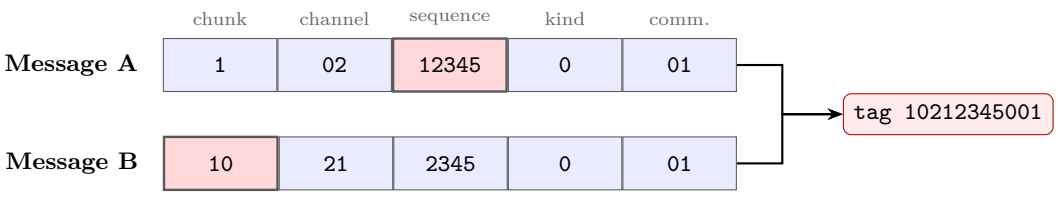


Upstream delivered every message to the sender's GPU index on the receiving host.

We added `peer_gpu_id` and changed endpoint selection to use `Gpu` at the source and `PeerGpu` at the destination. Queues and completion callbacks retain both endpoints, so matching checks the intended GPU pair and updates each host's own resource clocks. htsim's packet transport model stays the same.

Execution clocks use the explicit 32-bit `Stream` member rather than the 8-bit CPU field; the stream count comes from the schema. Legacy input retains its original routing; its adapter binds the CPU field to `Stream`.

Message matching also had to change. Previously, the matcher compared numeric tags containing concatenated components. Minimum-width decimal padding did not impose a maximum component size, so different component tuples could produce the same tag. For example, chunk 1, channel 2, sequence 12345 and chunk 10, channel 21, sequence 2345 (both with kind 0 and communicator 1) both produce the tag `10212345001`, because the five-digit sequence number shifts the other components:



Two different messages produce the same concatenated tag.

For explicit-field messages, both the waiting-receive and unexpected-message queues now compare the source and the complete `(Communicator, Channel, Sequence, MessageKind, Chunk)` tuple. The identity travels with the message through the network completion callback, so matching works whether the receive or message arrives first. Components remain unsigned 64-bit values throughout, avoiding narrowing or lossy replacement with a hash. If several queued operations match, the shared matcher selects the lowest operation index, preserving schedule order rather than relying on queue insertion order.

Legacy messages retain numeric-tag matching. Explicit-field messages require `tag 0`, message kinds keep the generator's values 0–5, and unmatched messages cause the run to fail. These checks prevent the two identity conventions from being mixed silently.

Compatibility boundary. The ordinary LogGOPSim matcher still supports numeric tags only, so its adapter rejects explicit ATLAHS messages. Running this particular trace in LogGOPSim would also require tuple-based matching there; accepting the file schema alone does not supply that behavior.

The modified ATLAHS source is a separate fork of a fixed upstream revision, cloned beside LogGOPSim; only the adapter lives in LogGOPSim. Build instructions are in the adapter README.

Integration results, including the complete recorded-workload comparison, are reported in Section 5.1.

4.4 Remaining conventions

Other conventions remain:

- ATLAHS also includes GOAL-generation paths for MPI/HPC workloads (`goal_gen/hpc`, using PMPI traces and Schedgen) and distributed-storage workloads (`goal_gen/storage`, using DirectDriveSim). We adapted only the NCCL generator. The others can still use the legacy simulation path, but their output cannot directly use our explicit-field ATLAHS adapter: their generators would first need to supply the required attributes with meanings appropriate to those workloads.
- GPU IDs are consecutive local indices (`0, 1, 2, ...`), which the modified NCCL generator assigns automatically from the recorded GPU identifiers. The simulator maps each host/GPU pair to a network endpoint using `host_rank` × `gpu_count` + `gpu_id`, where `gpu_count` is the file-wide maximum local GPU ID plus one. The topology must therefore provide enough endpoints for this mapping: two hosts with four GPUs each require eight endpoints. Our test topologies satisfy this requirement; custom traces and topologies must use the same mapping. Hosts with fewer GPUs still reserve endpoint slots according to the file-wide GPU count.

5 Evaluation

5.1 Correctness

The automated tests are split in two. **Unit tests** (`tests/unit/`, GoogleTest) check individual components in-process. **Integration tests** (`tests/integration/`, pytest) run the built programs end to end: converter, simulators, ATLAHS generator and the recorded workload. CTest runs both and labels them, so `ctest -L unit` runs only the fast tests; shared inputs are in `tests/data/`. The checks cover:

- **Format components:** header and rank-table round trips, byte order, packing at every bit alignment and width, wildcards, signed and custom fields, and rejection of malformed files.
- **Conversion:** new files are decoded independently and compared field by field with legacy output, including dependency lists, roots, wildcards, multiple custom fields and 64-bit values.
- **Specialized decoders:** a differential test generates 300 random layouts (widths 0–64, gaps, signed bases, wildcards, fields spilling into a ninth byte) with 2,000 random records each; 321,042 of the records are valid and 278,958 invalid. Every runner of the registry that handles any plan must accept exactly the records the exact decoder accepts and produce identical values, and the exact decoding into member destinations must agree as well; counter decrements must leave neighbouring bits unchanged. Removing the spill-byte load or the send-to-ANY check makes the test fail. Every runner, including the templated ones, also reads a trace that ends directly before a protected memory page. On all five benchmark traces, simulation results match the legacy format.
- **Simulation:** on a saved reference trace (`binomialtreebcast_8`), the new format must reproduce the original simulator's per-rank finish times. Further tests cover both message arrival orders and both protocols, a regression test for the eager/rendezvous bug described below, and 60 random schedules with

wildcards, start dependencies, several CPUs and NICs, mixed message sizes and custom fields, which must simulate identically in both formats under four network parameter sets.

Comparing the restructured simulator with the original also exposed an existing bug in LogGOPSim's eager/rendezvous handling. A small message is sent eagerly, without waiting for the receiver, and the sender completes immediately. If the message arrived before a larger receive was posted, which MPI allows, the receive side decided by its own buffer size that the message was rendezvous and completed the sender a second time. The original simulator let the dependency counter underflow silently and reported wrong times; in a two-rank example, a sender that should finish at 1,700 was reported at 100,000. The receive side now decides by the message's size, as the sender did.

Beyond the test suite, a one-off audit compared the new code with the original LogGOPSim on 1,480 schedules: 60 Schedgen collectives and 1,420 random graphs, each simulated with four network parameter sets. The legacy output of the new converter was byte-identical to the original's, and both formats reproduced the original simulator's results in all 5,920 simulations. This excluded the eager/rendezvous bug above. The original simulator crashed on two kinds of input: ranks without operations, which the new simulator handles, and Schedgen schedules that address non-existent ranks, which the new converter rejects with an error. The test suite also passes under AddressSanitizer and UndefinedBehaviorSanitizer, and 2,000 randomly corrupted files were each either rejected with an error message or simulated normally, without a crash.

5.1.1 ATLAHS integration

Small fixtures check individual behaviors; a recorded Llama workload checks the complete path from Nsight recordings through generation and conversion to htsim. The comparisons use htsim's NDP transport.

Check	Coverage	Result
Generator	Point-to-point, ring and tree AllReduce; two hosts with four GPUs each	Explicit output converts and simulates; default output is byte-identical to upstream.
GPU selection	Change NIC values independently of GPU IDs; vary GPU assignments	Explicit and legacy finish times agree. Changing GPU assignments affects timing.
Endpoint routing	GPU pairs 0→0, 0→2 and 2→0 on two four-GPU hosts, with both arrival orders	All six cases use the expected transport endpoints and complete; inconsistent peer GPU IDs are rejected.
Message matching	Distinct identity components, wide values, old tag collisions and both arrival orders	Matches the unique-tag legacy reference; invalid or unmatched messages are rejected.
Reuse and compatibility	Repeated simulation and upstream's NDP reference test	Input remains unchanged; reference test passes.
Recorded Llama workload (before `stream_id` and `peer_gpu_id`)	Four hosts, sixteen GPUs; 5,680,774 operations through the original and modified pipelines	All 475,452 matched message pairs and all four host finish times agree exactly.
Recorded Llama workload (with `stream_id` and `peer_gpu_id`)	The same workload with explicit partner GPUs and streams	All matched message pairs still agree; finish times are about 0.017% earlier, because messages now reach the partner GPU instead of the sender's index. The test no longer compares finish times.

Both recorded-workload pipelines disable benchmark-based reduction/copy timing. The reference uses the original converter to reproduce its historical 32-bit tag narrowing; our legacy converter rejects that overflow. The explicit-field path preserves the identity components.

5.2 File size

The table compares both formats for the benchmark traces of Section 5.4, a large Schedgen reduction and the recorded ATLAHS workload. Record sizes are per node; file sizes include the header, rank table, dependency lists and roots.

Trace	Nodes	Record	File (legacy → new)	Smaller
bcast_8	14	39 → 2 B	808 → 594 B	26%
bcast_1k	2,046	39 → 4 B	110 → 30 KiB	73%
bcast_64k	131,070	39 → 4 B	6.9 → 1.9 MiB	73%
wide_64k (random 64-bit values)	131,070	39 → 15 B	6.9 → 3.2 MiB	53%
alltoall_256	130,560	39 → 2 B	5.4 MiB → 404 KiB	93%
ring_256	261,120	39 → 4 B	10.7 → 1.3 MiB	88%
dissem_16k	458,752	39 → 4 B	19.2 → 2.4 MiB	88%
Schedgen binomial-tree reduce, 262,144 ranks	524,286	39 → 5 B	29.4 → 8.4 MB	71%
ATLAHS Llama recording	5,680,774	39 → 17 B	240.6 → 111.4 MiB	54%

Most Schedgen collectives shrink by 73–93%: their fields have narrow ranges or are constant, so records take 2–5 bytes instead of 39. On bcast_8, the header and rank table dominate the small file. With random 64-bit sizes and tags (wide_64k), records still need 15 bytes and the file is 53% smaller. The recorded Llama workload is 54% smaller while carrying all eight explicit ATLAHS fields, which the legacy format cannot store. Section 5.3 measures runtime.

5.3 Decoding performance

The initial measurements below predate the shared-memory-work improvements in Section 5.3.1. They compare the legacy format with the self-describing format, decoded by the original decoder (*generic*, now the exact fallback) and by both fast paths of Section 3.6 (*plan* and *JIT*). All runs used an Apple M3 Pro with clang `-O3` and report medians of 21 runs (15 for setup times); end-to-end times also give 95% confidence intervals after one warm-up run. Results varied by up to 30% between sessions on this laptop, so each table comes from a single session.

Method. The tool `decode_bench` replays exactly the reader accesses of a simulation, three node reads per node and one counter decrement per dependency edge, without the network model. Its overhead over legacy agrees with that of full simulations (28.6 vs. 35.3 ms on bcast_64k, 96.4 vs. 95.2 ms on dissem_16k), so the replay isolates decoding. The allreduce_1k trace is excluded from end-to-end runs because its simulation stops early on receives that never get a matching send, also with the original LogGOPSim; the replay still reads all its nodes.

How much time is decoding? With the generic decoder, decoding costs 190–270 ns per node, compared with 19–48 ns for legacy records:

Trace	Legacy (ms)	Generic (ms)	Decoding share
bcast_64k	51.7	87.0	40%
dissem_16k	147.7	242.9	47%
alltoall_256	1196	1248	2%

The share is replay time divided by simulation time. Decoding is up to half of the run time on collectives; alltoall_256 is dominated by message matching.

Per-node cost. Replay time per node:

ns per node	Legacy	Generic	Plan	JIT
bcast_64k	47.8	266.2	96.0	65.6
dissem_16k	40.7	250.9	84.0	53.6
alltoall_256	18.8	190.3	72.8	32.6
allreduce_1k	30.8	251.2	80.9	47.1

The plan is 2.6–3.1× faster than the generic decoder, and the generated decoder saves a further 30–40 ns per node (4.1–5.8× faster than generic). It remains 1.3–1.7× slower than legacy. At that stage, profiling attributed much of the remaining gap to repeated dependency-list validation and the indirect decoder call.

End-to-end time. Complete simulations, median and 95% confidence interval in milliseconds:

Trace	Legacy	Generic	Plan	JIT
bcast_64k	51.7 [51.0, 52.0]	87.0 [85.2, 91.2]	60.1 [58.6, 61.8]	57.0 [55.3, 59.5]
dissem_16k	147.7 [141.6, 156.9]	242.9 [234.7, 251.4]	146.0 [144.6, 147.9]	129.5 [127.8, 133.3]
alltoall_256	1196 [1166, 1249]	1248 [1203, 1285]	1243 [1208, 1256]	1237 [1187, 1255]

On bcast_64k the generated decoder is 10% slower than legacy, against 68% for the generic decoder. On dissem_16k it is 12% faster than legacy; our hypothesis is that the eight times smaller file (2.5 instead of 20 MB) causes fewer page faults, which remains to be verified. On alltoall_256 all decoders are within 5%.

Setup cost. Loading the bcast_64k schedule in a fresh process:

Decoder	Setup	Break-even vs. plan
Plan	4.2 ms	—
JIT, cached	3.9 ms	immediate
JIT, not cached	375.6 ms	about 12 million nodes
ORC	13.1 ms	about 300,000 nodes

The break-even point is the extra setup time divided by the saving per node (about 30 ns). Without the cache, compilation pays off only after about 12 million nodes; our largest trace has 4.2 million. Most of this time is not the compiler: in a separate run of 321 ms, compilation took 64 ms and the first `dlopen` 255 ms, because macOS verifies newly created binaries; later `dlopen` calls take about 1 ms. On Linux only compilation should count, giving a break-even of about 2.1 million nodes; this still needs to be measured. ORC compiles in memory on every run, which is cheaper than an uncached JIT by a factor of almost 30.

Predictions. Before implementing the decoders we predicted that per-node cost would order generic > plan > JIT with the JIT close to legacy; the order held, but the JIT remains 1.3–1.7× legacy. We predicted end-to-end times close to legacy on bcast and dissem (measured: +10% and −12%) and no visible change on alltoall_256 (confirmed). We did not anticipate the uncached setup cost.

5.3.1 Reducing shared memory work

The runners share the same node-reading and scheduling path. We removed repeated work there: graph setup validates dependency indices once, the reader resolves its type-erased validator once, and the scheduler reuses ready-node buffers instead of allocating a vector for each batch. List bounds and output-type checks remain on each read. These changes also reduce allocations in the legacy replay.

The current `decode_bench` generates its inputs through Schedgen and `txt2bin`. The table compares median replay time before and after these changes on the same machine, with nine repetitions per benchmark and `RelWithDebInfo` (`-O2 -g`). Values are nanoseconds per node; arrows indicate before → after:

Runner	bcast_64k	dissem_16k
Legacy	57.9 → 32.1	40.5 → 27.9
Plan	112.6 → 72.2	91.6 → 69.7
Templated node	93.2 → 59.0	77.1 → 56.5
Templated schema	72.7 → 39.1	57.7 → 35.8
C JIT	— → 39.3	58.6 → 36.2
LLVM ORC	76.9 → 40.5	58.5 → 36.9

The comparable self-describing replay cases use 24–47% less time per node. The broadcast C-JIT baseline was unavailable because its cache was not writable; its post-change value was measured separately with a writable cache. Reader setup does more work: the plan adapter increases from 0.35 to 0.56 ms on bcast_64k and from 0.09 to 0.43 ms on dissem_16k. The replay numbers exclude this setup cost.

We also reran `bench/end_to_end/decode_share.py` with 21 runs per configuration and a warm JIT cache. At that earlier stage, before list-only access, complete-simulation medians in milliseconds were:

Runner	bcast_64k	dissem_16k
Legacy	42.5	131.1
Plan	54.0	130.7
Templated node	46.5	125.4
Templated schema	43.1	112.1
C JIT	44.5	112.2
LLVM ORC	52.0	120.5

These timings include process startup, reading, scheduling and simulation. They compare runners at that earlier stage, rather than measuring an end-to-end before/after speedup. They do not establish results for other machines or cold caches.

For ATLAHS, we timed both pipelines on the recorded Llama workload (5,680,774 operations, one run each, wall-clock time). Simulation with htsim takes the same time: 133.6 s upstream and 134.0 s with explicit fields and the new format. Generating the GOAL text takes 16.9 instead of 10.9 s, because the generator writes eight more fields per operation, and conversion takes 19.5 instead of 6.5 s, because our converter reads the input twice and packs the extra fields (Section 3.3).

5.3.2 Reading only the required dependency list

LogGOPSim originally decoded a full node when it became ready, started and finished. Replacing the latter two reads with list-only access helped, but the first version still read both lists for each event. The scheduler now calls `read_dependents(rank, index, event)`, selecting `DependencyEvent::Start` or `DependencyEvent::Finish` before decoding. It reads only the selected list's position and count and builds one view. The legacy path makes the same selection.

The packed reader reuses `FieldSource`, resolving byte positions, shifts, masks and load extents at setup instead of maintaining separate extraction arithmetic. Zero-bit fields return their constant base without loading record bytes. Node indices and slice positions remain checked, including constant empty lists; wide fields and mapping tails keep their safe read paths. No dependency arrays are copied or allocated per read.

Replay includes dependency access, counter updates and ready-node handling, so it does not isolate decoder time. The changes were measured incrementally in one session, with five repetitions per case. Medians in ns per node, from the initial list-only implementation to the retained optimizations:

Runner	alltoall_256	bcast_64k	dissem_16k
Legacy	11.8 → 7.4	26.5 → 22.1	23.3 → 19.4
`plan`	25.4 → 11.1	47.6 → 37.1	42.8 → 32.4
`jit`	25.7 → 11.1	40.3 → 30.5	36.6 → 26.5
`orc`	25.8 → 11.1	40.2 → 30.7	36.6 → 26.4
`templated_node`	25.6 → 11.1	44.7 → 34.5	40.1 → 29.9
`templated_schema`	25.7 → 11.1	40.3 → 30.3	36.4 → 26.0

The event-specific change produced the largest general improvement; constant-field handling especially helped the dependency-free trace. Reusing `FieldSource` produced a smaller timing change while removing duplicated code. An additional empty-view shortcut was rejected: it improved replay slightly but slowed shuffled reads (JIT on dissem_16k rose from 14.4 to 18.7 ns per node and returned to 14.4 after removal). The retained changes improve the new format's replay by roughly 20–57% in these cases; legacy improves too and remains faster per node. Sampling points to dependency access and record lookup, but does not establish that memory stalls explain the remaining gap.

Complete simulations were also measured against the original LogGOPSim: the last upstream commit before this project (76451de), exported from the repository's history and built with its own CMake files and the same build type (target `loggopsim_original`). It simulates the legacy files; on all LogGOPSim benchmark traces except wide_64k, whose random tags leave messages unmatched and on which the original crashes, it produces the same output as our simulator in both formats. Each configuration ran 21 times, all configurations interleaved in a seeded random order to reduce drift. The C JIT cache was warm; ORC compiled in each process. Wall-clock medians, including startup and simulation, in milliseconds:

Trace	Original	Ours, legacy	plan	jit	orc	templ._node	templ._schema
bcast_8	3.1	3.3	3.3	3.6	12.8	3.4	3.2
bcast_1k	3.8	3.9	3.9	4.1	13.2	3.9	3.8
bcast_64k	48.4	43.9	45.1	43.7	52.9	44.1	43.6
dissem_16k	146.4	122.7	116.6	110.8	118.3	114.3	110.2
ring_256	39.8	32.8	34.5	32.9	41.8	33.6	32.4
alltoall_256	1950	2004	1997	1996	2010	1999	1998
wide_64k	crashes	78.4	79.6	79.5	88.3	81.3	77.8

On the three larger collectives, our simulator is 9–18% faster than the original with legacy files, and every runner except `orc`, which loads LLVM and compiles in each process (about 10 ms, visible on the small traces), is 7–24% faster with the new format. The gain comes from the restructured simulator rather than from the format: dependency indices are checked once, ready-node buffers are reused and starting or finishing a node reads only one list. On alltoall_256, where message matching dominates, all versions are at most 3% slower than the original; on the two small traces, process startup dominates. These numbers were measured after the reader refactoring of Section 3.5.2, so the legacy column uses the binding-based legacy reader. These are single-machine measurements; scheduler and network work, process startup and measurement variation also contribute to total time.

5.4 Preliminary benchmarks: which runner wins where

Different trace sizes and field distributions can favor different runners. The Google Benchmark suite (`bench/`) measures these cases on one laptop; the full benchmarking study on other machines remains outstanding.

Scenarios. The build generates eight traces, each of which varies one property. Record and file sizes compare the legacy format with the self-describing one (Section 5.2):

Trace	Nodes	Record	File	Property
bcast_8	14	39 → 2 B	808 → 594 B	Tiny trace: setup and compilation dominate
bcast_1k	2,046	39 → 4 B	110 → 30 KiB	Small trace
bcast_64k	131,070	39 → 4 B	6.9 → 1.9 MiB	Large trace, mostly constant (0-bit) fields
dissem_16k	458,752	39 → 4 B	19.2 → 2.4 MiB	Many dependencies per node
ring_256	261,120	39 → 4 B	10.7 → 1.3 MiB	Long dependency chains
alltoall_256	130,560	39 → 2 B	5.4 MiB → 404 KiB	No dependencies, many peers
wide_64k	131,070	39 → 15 B	6.9 → 3.2 MiB	bcast_64k with random values: no constant fields, 64-bit sizes and tags
atlahs_64k	131,070	25 B	4.5 MiB	bcast_64k with the eight ATLAHS custom fields, 19 fields per node, read through the ATLAHS adapter; no legacy file, which cannot store them

Benchmarks. For each trace and every runner of the registry, `decode_bench` measures *Setup* (creating the adapter, with a warm JIT cache), *ColdStart* (a one-shot run: setup with an empty cache, then reading every node once), *ReadNodes* (reading every node once in order), *RandomReads* (the same in a random order) and *Replay* (the simulator's accesses, as in Section 5.3). A script tabulates the results and marks the fastest runner, counting runners within 3% as tied. A sampling profiler script and Google Benchmark's statistical comparison support optimization work. The results are medians of three repetitions on an Apple M3 Pro, `RelWithDebInfo`.

Steady state. Decoding time per node when reading every node once (ReadNodes):

ns per node	plan	jit	orc	templated node	templated schema
bcast_8	30.2	17.8	17.9	24.1	17.4
bcast_1k	27.5	13.9	13.8	21.2	13.3

ns per node	plan	jit	orc	templated node	templated schema
bcast_64k	26.5	13.4	13.4	21.1	13.2
dissem_16k	29.2	12.9	12.8	24.0	12.4
ring_256	30.2	12.7	13.3	24.0	13.1
alltoall_256	29.0	12.4	12.4	24.1	11.9
wide_64k	26.4	13.7	13.8	21.1	13.5
atlahs_64k	41.2	17.2	16.6	29.9	16.3

The compiled runners and the known-schema template remain the fastest family for sequential node reads, with the node template between them and the interpreter. Additional ATLAHS fields increase the work per node. Random access costs more than sequential access, and is checked separately: an optimization that improves replay can still hurt a different read pattern.

Cold start. A one-shot run (setup with an empty cache, then one read of every node), in milliseconds:

ms	plan	jit	orc	templated node	templated schema
bcast_8	**0.006**	329	2.6	**0.006**	0.006
bcast_1k	0.074	334	3.0	**0.066**	0.048
bcast_64k	4.2	350	5.4	**3.5**	2.4
dissem_16k	13.9	345	**8.8**	11.6	6.1
ring_256	8.0	332	7.3	**7.0**	3.8
alltoall_256	3.8	347	4.4	**3.2**	1.6
wide_64k	4.2	342	5.7	**3.4**	2.4
atlahs_64k	6.1	338	6.1	**4.6**	2.8

Bold marks the fastest runner that works for any trace. `jit` spends 330–350 ms compiling and loading, so it never wins a cold start; ORC needs about 2.5 ms and wins on dissem_16k, the trace with the most nodes. Elsewhere `templated_node` wins because it needs no compilation; on the tiny bcast_8 it ties with `plan`. The best runner needs 1.4–2× the time of `templated_schema`. Setup with a warm cache is 0.004–0.8 ms for all runners.

Which runner wins where. No single runner is best everywhere:

- **Tiny traces** (bcast_8): `plan`; there is nothing to amortize.
- **One-shot runs up to a few hundred thousand nodes**: `templated_node`.
- **One-shot runs on large traces** (dissem_16k): `orc`.
- **Repeated runs and long simulations**: `jit` (with a warm cache) or `orc`, which tie.
- `templated_schema` matches the compiled runners without compiling anything, but only for layouts compiled in; here it covers the benchmark traces themselves. It serves as a reference, not as a choice for new traces.

A sensible automatic choice would therefore be `plan` for tiny traces, `jit` when its cache already holds the trace's decoder, and otherwise `templated_node` or `orc` by trace size. We have not implemented this yet.

5.5 Conversion time

We timed the complete `txt2bin` process on the same GOAL text for the benchmark traces: the original converter (built from the last upstream commit, as in Section 5.3.2), our converter writing the legacy format, and our converter writing the self-describing format. The original and our legacy converter produce byte-identical files. Medians of five runs, all interleaved in a seeded random order, warm input cache, in milliseconds:

Trace	Original	Ours, legacy	Ours, self-describing
bcast_8	53	51	3.3
bcast_1k	63	119	7.3
bcast_64k	758	4527	247
dissem_16k	362	1462	333
ring_256	138	187	252
alltoall_256	74	136	82
wide_64k	779	4571	295
atlahs_64k	n/a	n/a	585

The legacy format cannot store atlahs_64k's custom fields. Despite reading the input twice, the self-describing converter is faster than the original on five of the seven traces and slower on ring_256 (1.8×) and, with its eight custom fields, on the recorded ATLAHS workload (19.5 instead of 6.5 s, Section 5.3.2). Both legacy converters end with a system-wide `sync()`, which accounts for about 50 ms even on the eight-rank trace; the self-describing writer closes its stream without one. Our legacy writer is up to 6× slower than the original because it grows and remaps the output file for every rank, which dominates on traces with many ranks; this is a defect of our writer, not of the format, and remains to be fixed.

5.6 Remaining measurements

The study above covers decoding on one laptop. The full benchmarking study is still outstanding, including:

- repeating all measurements on Euler (Linux, x86-64), including uncached JIT setup;
- peak memory of `txt2bin` for both formats, and conversion time on Linux;
- complete file sizes for bit-packed and byte-aligned layouts across schemas, including custom fields;
- peak simulation memory, and runs with cold file caches.

6 Limitations and future work

Cheaper conversion. Both passes currently run the full parser and build complete graphs. Section 3.3 compares lighter passes with alternatives that avoid the second text parse. Implementing and comparing these designs for conversion time, peak memory and temporary storage remains future work.

Better compression. Dictionary encoding, described in Section 3.3.1, remains a proposed alternative to offsets. Separately, the start-dependency position `sDepIx` always equals `depIx + nDep` and could be derived instead of stored.

Richer metadata. The header does not yet describe units, which operations a field applies to, default values or provenance. A canonical schema hash would let decoders be cached by layout.

Standard field limits. Standard fields are still limited by the parser and simulator types they are stored in, for example 32-bit tags and 8-bit CPU IDs; widening them requires changing those paths together, not only the file layout.

Remaining decoding overhead. Dependency indices are validated once at graph setup, ready-node buffers are reused, and starting or finishing a node reads only its dependency lists (Section 5.3.2). Excluding the 14-node trace, current replay medians span roughly 13–41 ns per node against 7–27 ns for legacy records; profiling points to dependency-list access and record lookup. The unused list reads are now removed. Repeated rank lookup, packed-index iteration and counter updates remain candidates for measurement; sampling alone does not establish memory latency as the cause.

Decoder selection. The interpreted runner is the default and the others are selected with an environment variable; a command-line option would be more convenient, and Section 5.4 suggests choosing the runner automatically. Since a failed compilation or a missing template instantiation falls back to the interpreter, the adapters report which runner is in use (`get_runner_name`); LogGOPSim prints it, and the simulation tests run with every runner.

Evaluation. The measurements in Sections 5.3–5.5 come from one laptop; Section 5.6 lists what remains.

Parallelization. The simulator and converter are single-threaded. OpenMP parallelization and further optimizations are planned.

7 Conclusion

The project replaces LogGOPSim's fixed 39-byte binary trace with a self-describing format whose header names every field and gives its type, width and position. Producers can add custom fields without changing the format, readers validate at load time that a file supplies what each simulator needs, and fields are packed into only as many bits as their values require. The ATLAHS case study makes GPU and message identity explicit, replacing overloaded NICs and concatenated tags with validated fields and tuple-based matching. Correctness is established by the test suite against the original simulator. Runtime specialization recovers most of the decoding overhead: a decoder compiled for the file's schema decodes nodes 4–6× faster than the generic decoder, with the current complete-simulation results reported in Section 5.3.1. Five runners share one decode plan behind a registry, and preliminary scenario benchmarks show that the best one depends on trace size and whether a run is one-shot or repeated (Section 5.4). The broader performance evaluation remains to be completed.

8 Appendix: build and conversion

The header format is specified in `doc/binary_header.md`. The project builds with CMake 3.20 or newer and C++23; the Makefile delegates to CMake. The tests run with `ctest` from the build directory; `ctest -L unit` runs only the fast unit tests and `ctest -L integration` the end-to-end checks. The benchmarks are in `bench/` and are described in `bench/README.md`; each runner's folder and how to add one are described in `src/LogGOPSim/binary_reading/decoding/runners/README.md`. Convert a trace with:

```
./txt2bin -i example.goal -o example.bin --format self-describing --report-schema
```

The report source is `report/project_report.tex`; edit it directly and rebuild with `./report/build.sh`. Figures are maintained in `report/diagrams/`.